

Playwright Locator Filters

What are Locator Filters?

Locator filters help you **narrow down elements** when multiple elements match the same locator.

Instead of selecting all matching elements, filters allow you to:

- Pick specific elements
- Exclude elements
- Combine multiple conditions

They make your tests more **accurate and readable**.

1. hasText

What it does:

Filters elements that **contain specific text**.

Example HTML:

```
<ul>
  <li>Apple</li>
  <li>Banana</li>
  <li>Apple Juice</li>
</ul>
```

Example:

```
const items = page.locator('li').filter({ hasText: 'Apple' });
```

Result:

- Matches: "Apple", "Apple Juice"

When to use:

- When multiple elements exist and you want those containing certain text

2. hasNotText

What it does:

Filters elements that **do NOT contain specific text**.

Example:

```
const items = page.locator('li').filter({ hasNotText: 'Apple' });
```

Result:

- Matches: "Banana"

When to use:

- When you want to exclude certain elements

3. first(), last(), nth()

These methods help select elements based on **position**.

first()

What it does:

Selects the **first matching element**

```
const firstItem = page.locator('li').first();
```

last()

What it does:

Selects the **last matching element**

```
const lastItem = page.locator('li').last();
```

nth(index)

What it does:

Selects element at a specific index (0-based)

```
const secondItem = page.locator('li').nth(1);
```

Important:

- Index starts from **0**
- nth(0) is the first element

When to use:

- When order of elements matters
- When elements do not have unique identifiers

4. Matching Two Locators (AND condition)

locator.and()

What it does:

Matches elements that satisfy **both conditions**

Example:

```
const button = page.getByRole('button')
    .and(page.getByText('Submit'));
```

Result:

- Finds a button that has text "Submit"

When to use:

- When one locator is not enough
- When you want more precise matching

5. Matching One of Two Locators (OR condition)

locator.or()

What it does:

Matches elements that satisfy **either condition**

Example:

```
const element = page.getByText('Login')
    .or(page.getByText('Sign in'));
```

Result:

- Matches either "Login" OR "Sign in"

When to use:

- When UI may have different text

- When supporting multiple scenarios or environments

Summary

- `hasText` → include elements with text
- `hasNotText` → exclude elements with text
- `first()`, `last()`, `nth()` → select by position
- `locator.and()` → match both conditions
- `locator.or()` → match either condition

Tips

- Use filters when locators return **multiple elements**
- Prefer readable combinations instead of complex selectors
- Avoid overusing `nth()` unless necessary
- Combine filters with role/text locators for better stability